

CampusDesk: why student support tickets take so long, and what to fix first

Lakshya Mewara · Roll No. 24bcs10290 · Batch 2024-28

GitHub repository [GitHub repository link \(to be added\)](#)

5-min demo video [5-min demo video \(Google Drive\) link \(to be added\)](#)

Data Synthetic, realistic, reproducible (seeded generator in the repo). No real student data used.

About the project. A college's Dean of Student Affairs says: *"Students keep complaining that helpdesk tickets take forever. Before we buy an AI chatbot, find out what is actually happening."* I investigated it as a Forward Deployed Engineer. I identified the stakeholders, mapped the ticket workflow and wrote a SMART problem statement with one KPI. I then pulled data from four systems (SQL ticket DB, CSV audit log, JSON survey export, paginated REST API), profiled and validated it with business rules, and modelled the ticket journey. Finally I built a repeatable pipeline that retrieves, validates, models and publishes the metrics, and refuses to publish when the data is not safe.



Answer first (pyramid)

Don't buy the AI chatbot yet. The delay isn't in answering students. It's in how tickets are routed, queued and escalated. Fixing intake routing and moving escalation to 50% of the SLA is estimated to lift compliance from 63% to about 79% (with a priority-first queue). This uses the existing team and tool. It should be piloted for 2 weeks before rolling out.

- **Priority is captured but ignored:** High-priority tickets wait as long for a first response (19.3h median) as Low (16.3h), so only 21% meet 24h.
- **Misrouting compounds delay:** 12% of tickets are reassigned. SLA falls 66% → 51% → 18% with each hop, and follow-ups rise 1.9 → 5.7.
- **Two departments hold the backlog:** Fees & Finance 28% and Academics & Exams 45% (weekday office hours only), against IT 87%.
- **Escalation comes too late:** a daily review escalates tickets open >48h (median 65h). That is after the deadline for every High ticket.

What is in this PDF

1	One-page problem brief: stakeholders, workflow, problem, KPI, scope	p.2
2	FDE discovery: stakeholders, goals, constraints, facts, assumptions, unknowns	p.3
3	Workflow & problem structure: current-state map, issue tree, hypotheses	p.4
4	KPIs, scope & success	p.5
5	Client data & source systems: source map, source of truth, key fields, missing events	p.6
6	Retrieval: 4 source types, reproducible, raw preserved	p.7
7	Profiling & cleaning: 15 business rules, safe fixes, assumptions	p.8–9
8	Workflow model with data: entities, events, interventions, outcomes, SQL metrics	p.10–11
9	Findings & recommendation	p.12–14
10	Dependable pipeline: run log, checks, failure handling, reliability evidence	p.15–16
11	What remains unknown · 5-min demo script	p.17

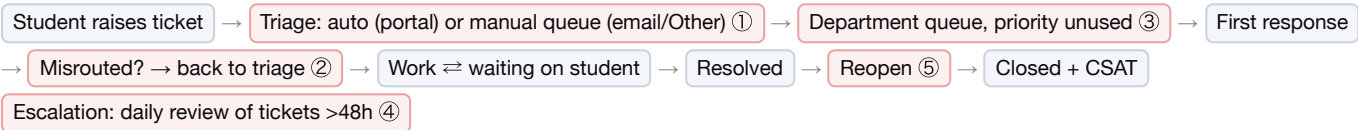
1 One-page problem brief

Client: Student Services Helpdesk of a residential engineering college (about 1,800 students; 15 helpdesk staff across 5 departments plus triage). **Sponsor:** Dean of Student Affairs. **Original request:** "Buy an AI chatbot because tickets take forever." **FDE reframing:** find where the time actually goes first.

Stakeholders

Who	Role in problem	What they want
Dean of Student Affairs	Sponsor, owns the KPI	Fewer complaints; a defensible decision on the chatbot spend
Helpdesk Manager	Operational owner	Know which queue or step is slow; fair workload
Triage coordinators (2)	Route email/'Other' tickets	Less manual sorting
Department agents (IT, Hostel, Finance, Academics, Library)	Resolve tickets	Receive only tickets that belong to them
Students	Affected users	Fast, first-time-right resolution; visibility
IT admin	Owns ticketing tool data	No disruption to the tool

Current workflow (where the problem appears)



Before the problem: intake and categorisation decide the first queue. After the problem: students chase with follow-ups, reopen tickets and give low CSAT.

Problem statement (SMART)

Only **63%** of student helpdesk tickets resolved between 27 Jul and 20 Sep 2026 met their priority SLA (High 24h / Medium 72h / Low 120h), and only **21%** of High-priority tickets did. **Identify where in the ticket workflow delay accumulates** and implement the highest-impact process fixes to raise SLA compliance to **≥75%** (High-priority to **≥50%**) **within 8 weeks**, using the **existing helpdesk team and current ticketing tool** (no new hires, no new software).

KPI & supporting metrics

Metric	Baseline
KPI: SLA compliance rate (resolved within priority SLA, net of waiting-on-student)	63.1%
M1 Median time to first response	17.9h
M2 Median manual-triage wait	13.2h
M3 Reassignment rate	12.4%
M4 Reopen rate	9.4%
M5 Average CSAT (1–5), response rate	3.69 (36%)

Scope

MVP (in): all tickets from the 3 intake channels over 8 weeks; the lifecycle from creation to final resolution; routing, queueing, escalation and reminders; a repeatable pipeline and KPI report.

Out of scope: building or buying the chatbot; staffing and HR changes; agent-level performance ranking; real-time dashboards; changes to the ticketing tool's code.

Success means: the KPI is **≥75%** and High **≥50%** on the same, signed-off definition, with reassignment rate and median first response falling. Early read on leading indicators within 2 weeks of the pilot.

Data: synthetic, realistic (seeded generator). Sources: SQLite ticket DB, CSV audit log, JSON CSAT export, paginated Portal REST API.

2 FDE mindset & stakeholder discovery

The request was a **solution** ("buy an AI chatbot"). Following the FDE loop, *Listen* → *Observe* → *Question* → *Structure* → *Validate*, I first asked who is affected, who owns the problem, and what "takes forever" actually means. The discovery questions below are the ones I would take into the first stakeholder interviews.

Stakeholder	Goal	Constraints	Discovery questions	What changes if they're right
Dean of Student Affairs (sponsor, KPI owner)	Fewer complaints; decide on the chatbot budget	Must show improvement before end-sem exams; limited budget	What would convince you the helpdesk has improved? Which complaints reach you?	Defines the KPI and target; signs off the SLA definition
Helpdesk Manager (operational owner)	Hit SLAs with current staff	No new hires; tool is fixed	Which tickets do you escalate, and when? How are priorities used in queues?	Owns routing and escalation rules, and confirms category mappings
Triage coordinators	Route fast and correctly	2 people, Mon–Sat 9–18, no Sunday cover	Why do tickets bounce back? What information is missing at intake?	Source of misrouting insight (no reason code exists)
Dept agents (IT, Hostel, Finance, Academics, Library)	Only receive their own tickets	Finance and Academics work Mon–Fri 9–17	How do you pick the next ticket? What does "waiting on student" mean for you?	Explains queueing (FIFO?) and the pause usage
Students (affected)	Quick, first-time-right fix	Exams, hostel move-in peaks	What made you follow up or reopen?	Validates that follow-ups and CSAT reflect real pain
IT admin (data owner)	Keep the tool stable	Read-only access for us	Which export is authoritative for timestamps? Any clock or sync issues?	Source-of-truth decisions; fixes to the walk-in desk clock

Facts (verified in data)

- 1,230 unique tickets in 8 weeks; 3 intake channels
- Draft SLA: High 24h / Medium 72h / Low 120h
- Email tickets have no category; they go to manual triage
- Escalation happens at a daily 10:00 review of tickets open >48h
- Auto-reminder to the agent after 24h without a reply

Assumptions (need confirmation)

- SLA clock pauses while "Waiting on Student"
- SLA runs to the **final** resolution (after reopens)
- Missing priority (email) = Medium, per policy default
- "Exam Cell" = Academics & Exams; "Hostel" = Hostel & Maintenance
- Cancelled and no-reply closures are excluded from the KPI

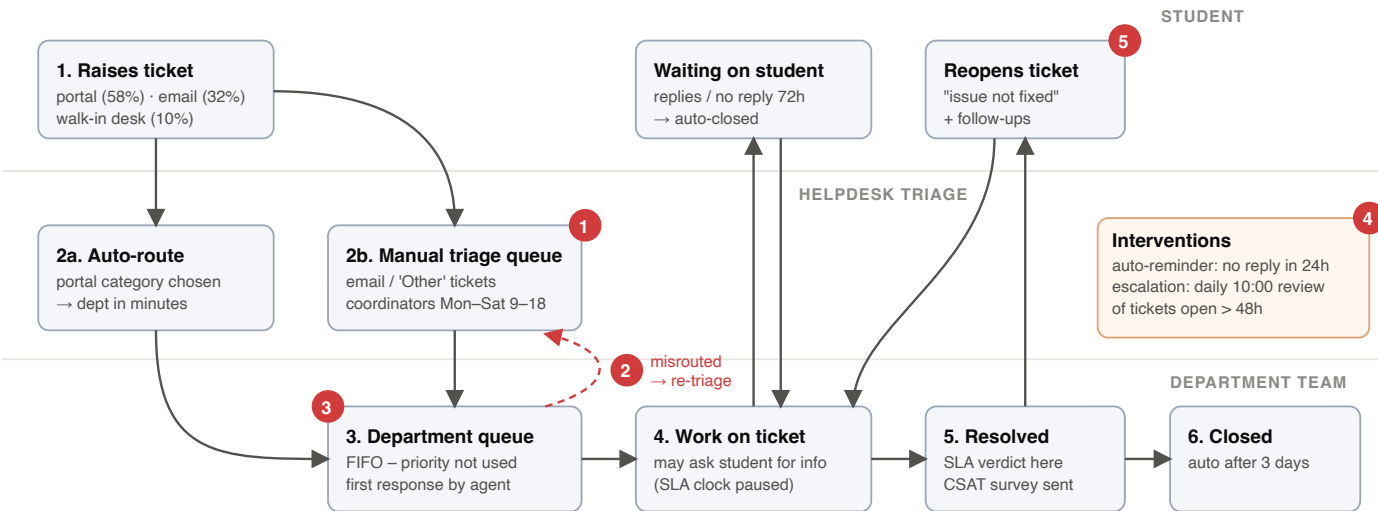
Unknowns (cannot get from data)

- **Why** tickets are misrouted (no reason code)
- Real agent effort (no time tracking)
- Causal effect of escalation (only slow tickets get escalated)
- Calendar vs business-hours SLA: not decided
- Views of students who don't answer the CSAT survey

FDE judgement: a chatbot answers students faster, but the data shows first contact isn't the bottleneck. Queueing, routing and escalation are. Buying it now would be "building the request" instead of solving the problem.

3 Workflow & problem structure

Current-state workflow (pain points ①–⑤ found in the data)



#	Where the problem appears	Before it	After it
①	Manual triage queue for email/'Other' tickets (median wait 13.2h)	Student sends an email with no category	Ticket reaches a department late
②	Misrouting: wrong department → back to triage (12.4% of tickets)	Wrong category chosen at intake or triage	Each hop adds about a day; more follow-ups and reopens
③	Department queue ignores priority (median first response High 19.3h vs Low 16.3h)	Priority is captured at intake	High-priority tickets miss 24h
④	Escalation only at a daily review of tickets >48h	Ticket is already slow	132 tickets escalated after the SLA had already passed
⑤	Reopen after a "quick fix" (9.4% of tickets)	Resolved without confirmation	The clock keeps running to the final fix

Issue tree: "Why do tickets miss SLA?" (MECE by workflow stage)

Branch	Sub-issues
A. Before a team owns it	A1 triage wait · A2 wrong first department (misrouting)
B. Waiting in the team queue	B1 priority not used · B2 team capacity / working hours · B3 volume peaks (move-in, exams)
C. While being worked	C1 work duration · C2 waiting on student (paused)
D. After "resolution"	D1 reopen cycles
E. Safety net	E1 reminders and escalations too late or ineffective

These branches add up exactly: the model splits each ticket's total time into triage + first-response wait + re-routing + work + waiting-on-student + reopen cycle. They are mutually exclusive and collectively exhaustive, and verified in code (max gap 0.00h).

Hypotheses to test (and what would disprove them)

Hypothesis	Test
H1 Misrouting / manual triage drives breaches	SLA by reassignment count and triage path
H2 Priority is not used in queueing	First-response time and SLA by priority
H3 Some departments are capacity or hours constrained	SLA and first-response time by resolving department
H4 Escalation comes too late to help	Escalation timing vs SLA deadline
H5 Weekend / email tickets are the main problem	SLA by weekday vs weekend and by channel

Prioritised on impact × feasibility: H1–H4 can be fixed with process and config changes inside the current tool (high feasibility). Capacity (B2) needs staffing, which is out of scope.

4 KPIs, scope & solution

Project KPI (lagging): SLA compliance rate

Definition: resolved tickets whose *net* resolution time (creation → **final** resolution, minus time in "Waiting on Student") is within the priority target, ÷ all resolved tickets in the period. **Grain:** ticket. **Population:** resolved in period. Excludes cancelled/duplicate, closed-no-reply, still-open and clock-skewed tickets (all counted and disclosed). **Baseline:** 63.1% (675/1069). **Target:** ≥75% in 8 weeks.

Supporting metric	Formula	Type	Why it matters / link to KPI	Baseline
M1 Median first-response time	median(first_response – created)	Leading · workflow driver	Queueing delay before anyone looks at the ticket	17.9h
M2 Median manual-triage wait	median(first_assigned – created) for manual tickets	Leading · driver	Pain point ①; fixed by structured intake	13.2h
M3 Reassignment rate	% tickets with ≥1 reassignment	Leading · driver	Pain point ②; each hop cuts SLA odds	12.4%
M4 Reopen rate	% resolved tickets reopened	Quality outcome (guardrail)	Stops "fast but wrong" fixes gaming the KPI	9.4%
M5 Average CSAT	mean(latest valid rating per ticket)	Student outcome (lagging)	Confirms students feel the change (directional: 36% response)	3.69

Also tracked as a guardrail: **High-priority SLA compliance** (baseline 21%, target ≥50%), so the headline KPI can't improve by only fixing easy Low tickets.

MVP scope (in)

- All tickets from portal, email and walk-in; 27 Jul – 20 Sep 2026
- The full lifecycle from the audit log; interactions from the Portal API; CSAT
- Diagnosis of where delay accumulates; sizing of process fixes
- Repeatable pipeline: retrieve → validate → model → metrics, with a gate and logs
- Recommendation plus a pilot plan with leading and lagging indicators

Out of scope

- Building or buying the AI chatbot (revisit after the pilot)
- Hiring and staffing changes; agent-level performance ranking
- Real-time queue dashboard (data is a daily batch export)
- Changing the ticketing tool's code; predicting ticket volume
- Non-helpdesk channels (WhatsApp groups, direct calls to wardens)

What success means

Horizon	Indicator	Success threshold
Weeks 1–2 (pilot, leading)	Reassignment rate · manual-triage wait · % tickets escalated before 50% of SLA	Reassignment <6% · triage wait <2h · >90% escalated on time
Week 8 (lagging)	SLA compliance · High-priority SLA · reopen rate · CSAT	≥75% · ≥50% · no increase · no decrease
Always	Definition signed off by the Dean; pipeline gate PASS or known WARNs only	Same definition before and after, otherwise the comparison is invalid

5 Client data & source systems

Data choice: synthetic, realistic, fully reproducible. Real student-helpdesk data is private, so generator/generate_data.py (seed 42) **simulates the workflow** (arrival peaks at move-in and mid-sem exams, working hours per team, manual triage queue, misrouting with 85% triage accuracy, FIFO department queues, waiting-on-student pauses, reopens, a 24h auto-reminder, the 48h daily escalation review, CSAT with non-response bias). It then writes the data **the way the real systems would hold it**, with realistic defects injected. The simulated ground truth is kept separately and used only to prove the pipeline is correct (section 10).

Information needed	Where it lives	Type	Source-of-truth decision
Ticket header: student, channel, intake category, priority, subject	helpdesk.db :: tickets	SQL	Source of truth for header fields
Student attributes	helpdesk.db :: students	SQL	Source of truth (names/emails not pulled)
Agents and departments	helpdesk.db :: agents	SQL	Source of truth
Lifecycle timestamps (created, assigned, first response, waiting, resolved, reopened, closed)	ticket_events.csv (audit-log export)	CSV	Source of truth for all times; the ticket table's resolved_at is stale (R12)
Interventions (escalations, auto-reminders)	ticket_events.csv	CSV	Source of truth
Student follow-ups, agent replies, info requests	Portal REST API /api/v1/interactions	API	Source of truth for interactions
Satisfaction rating 1–5	csat_survey_export.json (nested)	JSON	Directional only (response bias)
SLA targets and rules	sla_policy.json	JSON	DRAFT, not signed off (R15)

Key fields

Entity	Key fields
Ticket	ticket_id, student_id, channel, category, priority, status, created_at, resolved_at
Event	event_id, ticket_id, event_time, event_type, from_status → to_status, department, actor_id
Interaction	interaction_id, ticket_id, kind (follow_up / agent_reply / info_request / reopen_comment), created_at
CSAT	answers.ticket_ref, answers.rating, submitted_at

Injected defects (what real exports look like)

- Exact duplicate ticket rows (importer double-sync) and double submissions
- Mixed timestamp formats: ISO vs DD/MM/YYYY HH:MM from the email importer
- 20 spellings of 5 categories; 11 spellings of 6 statuses
- Missing priority on about a third of email tickets
- Stale denormalised resolved_at (first, not final, resolution)
- Duplicate and orphan audit events; walk-in desk clock skew
- CSAT: 4 ticket-ref formats, string ratings, out-of-range values, resubmissions
- API: pagination, random HTTP 500 and 429 (rate limit)

Important missing data / events

Missing	Why it matters	Instrument next
Reassignment reason code	We see <i>that</i> tickets bounce, not <i>why</i>	Mandatory drop-down on reassign
Agent "started work" event and time spent	"Work" time mixes queueing inside the team and real effort	"Start work" status plus effort field
Priority on email tickets	129 resolved tickets use a defaulted SLA target	Importer maps priority, or triage sets it
Student confirmation of fix	Reopens are only noticed when the student complains	"Did this fix it?" button before closing

6 Retrieval: four source types, reproducible, raw preserved

pipeline/retrieve.py pulls every source into an **immutable raw snapshot** data/raw/<run_id>/ before anything is cleaned. It writes _manifest.json with row counts and sha256 checksums, so every published number traces back to exactly what the systems returned on that run. The SQL source is opened **read-only**. Only needed columns are pulled; student names and emails are not.

Source type	Files saved	Rows
API	23	4550
CSV	1	7849
JSON	2	425
SQL	3	3069

Reproduce

```
python generator/generate_data.py
python pipeline/run_pipeline.py --run-id demo_run
```

The API is a local mock (api/mock_portal_api.py, standard library only). It is paginated and seeded to fail about 15% of requests with 500 and about 7% with 429 + Retry-After, like a real SaaS API. The pipeline starts it automatically.

The API: "HTTP 200 on one page does not prove the job succeeded"

```
pipeline/retrieve.py
def _get(url: str, log) -> dict:
    """GET with retries: 5xx and 429 are retryable, 4xx is not."""
    for attempt in range(1, MAX_RETRIES + 1):
        try:
            with urllib.request.urlopen(url, timeout=10) as r:
                return json.loads(r.read())
        except urllib.error.HTTPError as e:
            if e.code == 429:
                wait = float(e.headers.get("Retry-After", 1))
            elif 500 <= e.code < 600:
                wait = 0.2 * 2 ** (attempt - 1)
            else:
                raise PipelineError(f"Non-retryable HTTP {e.code} for {url}")
            if attempt == MAX_RETRIES:
                log.error(f"API HTTP {e.code} on attempt {attempt}/{MAX_RETRIES} -> giving up ({url})")
                break
            log.warning(f"API HTTP {e.code} on attempt {attempt}/{MAX_RETRIES} -> retry in {wait:.1f}s ({url})")
            time.sleep(wait)
```

```
pipeline/retrieve.py
log.info(f"API {total_pages} pages, {fetched} records fetched, API reports total_records={total_records}")
# Completeness proof: HTTP 200 on each page is not enough.
if fetched != total_records or len(ids) != total_records:
    raise PipelineError(f"API ingestion incomplete: fetched={fetched}, unique={len(ids)}, expected={total_records}")
```

Actual run log (retrieval stage): 14 transient failures retried, all 4,550 records proven complete

```
18:41:48 | INFO | retrieve | SQL tickets 1254 rows <- SELECT * FROM tickets
18:41:48 | INFO | retrieve | SQL students 1800 rows <- SELECT student_id, program, year, hostel_block FROM students
18:41:48 | INFO | retrieve | SQL agents 15 rows <- SELECT agent_id, department, role, active FROM agents
18:41:48 | INFO | retrieve | CSV events 7849 rows <- ticket_events.csv
18:41:48 | INFO | retrieve | JSON csat_surv 424 rows <- csat_survey_export.json
18:41:48 | INFO | retrieve | JSON sla_polic 1 rows <- sla_policy.json
18:41:48 | WARNING | retrieve | API HTTP 429 on attempt 1/6 -> retry in 1.0s (page 2)
18:41:49 | WARNING | retrieve | API HTTP 500 on attempt 1/6 -> retry in 0.2s (page 3)
18:41:50 | WARNING | retrieve | API HTTP 500 on attempt 1/6 -> retry in 0.2s (page 5)
18:41:50 | WARNING | retrieve | API HTTP 500 on attempt 1/6 -> retry in 0.2s (page 6)
18:41:50 | WARNING | retrieve | API HTTP 500 on attempt 1/6 -> retry in 0.2s (page 7)
18:41:50 | WARNING | retrieve | API HTTP 500 on attempt 2/6 -> retry in 0.4s (page 7)
... 8 more transient failures retried ...
18:41:54 | INFO | retrieve | API 23 pages, 4550 records fetched, API reports total_records=4550
18:41:54 | INFO | retrieve | raw snapshot written to data/raw/demo_run (29 files, manifest + checksums)
```

7 Profiling & cleaning

Business-oriented quality rules → validation gate: **PASS WITH CAVEATS**

Each rule is a business assumption, tested on every run. **FAIL** stops the pipeline. **WARN** means handled and disclosed. **UNKNOWN** means only a business owner can decide. This run: 6 PASS · 8 WARN · 1 UNKNOWN · 0 FAIL, so the KPI is published as **PROVISIONAL** with caveats. Evidence strings are generated by the pipeline itself.

#	Business rule (assumption)	Status	Evidence (this run)	Action	Owner
R01	Every source returned data; API returned all records	PASS	empty sources: none; API completeness proven in retrieve stage	stop run if any source empty	FDE / data
R02	One row = one ticket (ticket_id unique)	WARN	24 exact duplicate rows (1.9%); 0 conflicting ids	exact duplicates dropped (safe); conflicting ids would stop the run	FDE / data
R03	A student's repeat submission of the same issue is not counted twice	PASS	22 same student+subject within 15 min; 22 already marked Cancelled	Cancelled tickets excluded from KPI; the rest are NOT auto-merged (could be genuine repeats)	Helpdesk Manager
R04	created_at is a valid timestamp	WARN	200 rows in DD/MM/YYYY format (email importer); unparseable after fix: 0.0%	parsed both formats; day-first confirmed against the audit log (R06)	FDE / data
R05	Every audit-log event belongs to a known ticket	WARN	40 orphan events (0.5%); 77 duplicate events dropped	orphans quarantined, not joined	IT admin (ticketing tool)
R06	Every ticket has a 'created' event and both systems agree on created_at	PASS	tickets with audit-log lifecycle: 100.0%; created_at agreement within 1 min: 99.3% (DD/MM rows agree only when parsed day-first)	source of truth for lifecycle times = audit log; stop run if lifecycle coverage < 98%	FDE / data
R07	'created' is the first event of every ticket (nothing happens before creation)	WARN	8 tickets (0.7%) have events before 'created' (6 even resolved before creation); channels: {'walk_in': 8} -> walk-in desk clock skew	excluded from KPI (not guessed); walk-in desk clock to be fixed	IT admin (walk-in desk PC)
R08	Each ticket maps to one canonical category	WARN	20 raw values -> 5 canonical + Unknown; representation fixes=1088, semantic mappings=142 (Academics, Exam Cell, Fees, Finance, HOSTEL, Hostel, IT), Unknown/Other=41	semantic mappings applied provisionally; analysis uses RESOLVING department from audit log instead	Helpdesk Manager
R09	Every status maps to a known lifecycle state	PASS	11 raw values -> 6 canonical; unmapped=0	casing/underscore variants normalised	FDE / data
R10	Every ticket has a priority (it sets the SLA target)	WARN	145 tickets (11.8%) missing priority, all email channel	defaulted to 'Medium' per draft policy; KPI sensitivity reported	Helpdesk Manager
R11	Tickets map to students; interactions map to tickets	PASS	tickets->students 100.0%; interactions->tickets 100.0%	none needed	FDE / data
R12	resolved_at in ticket table = final resolution in audit log	WARN	of 1081 resolved tickets: 42 missing resolved_at, 99 hold the FIRST resolution (reopened)	use audit log as source of truth; ticket-table resolved_at ignored	IT admin (ticketing tool)
R13	One valid 1-5 rating per ticket	WARN	424 responses: 6 no ticket ref, 9 out of 1-5, 15 resubmissions (kept latest) -> 394 valid; ref formats normalised (tk100234 / 100234 -> TKT-100234); maps to tickets 100.0%	directional only: ~36% response rate, unhappy students respond more	FDE / data
R14	Data is fresh enough for the decision	PASS	latest event 2026-09-21 05:45; API last_synced 2026-09-21T06:00 (6h after period end)	fit for weekly/term reporting; NOT fit for a live queue dashboard (daily batch export)	FDE / data
R15	SLA definition is agreed by the KPI owner	UNKNOWN	policy file says: 'Student Helpdesk SLA v1 (draft, not formally signed off)'; pause rule & calendar-vs-business hours not signed off	publish as PROVISIONAL; show sensitivity to definition	Dean of Student Affairs

Profile of the raw snapshot

Source	Rows	Notable
tickets (SQL)	1,254	1,230 distinct ticket_id → duplicates; nulls: priority 11.8%, resolved_at 15.4%
audit events (CSV)	7,849	8 event types; unordered export; nulls by design in from/to status
interactions (API)	4,550	4 kinds; all map to tickets
CSAT (JSON)	424	nested respondent / answers; ratings as int <i>and</i> string

Raw category values: IT Support (293), Hostel & Maintenance (234), Academics & Exams (199), Fees & Finance (114), Library (90), Other (41), IT support (34), Hostel (33), IT (31), academics & exams (23), hostel maintenance (22), HOSTEL (21), it support (21), Exam Cell (19), Finance (15), library (14), fees & finance (13), Academics (13), Library (12), Fees (12)

Raw status values: Closed (923), closed (69), CLOSED (61), In Progress (60), Resolved (42), Assigned (38), Closed – No Response (25), Cancelled (22), in_progress (7), resolved (4), closed_no_response (3)

Safe fixes applied (representation only: they don't change business meaning)

- Dropped 24 exact duplicate ticket rows (email importer double-sync).
- Parsed 200 DD/MM/YYYY HH:MM timestamps (email importer) alongside ISO format.
- Dropped 77 duplicate audit-log events (same event_id).
- Normalised category casing/whitespace (e.g. 'it support', 'HOSTEL ' -> canonical).
- Normalised status variants (e.g. 'closed', 'in_progress', 'closed_no_response').
- CSAT: normalised ticket refs, cast string ratings to int, dropped out-of-range, kept latest per ticket.

Deliberately NOT "cleaned by instinct"

Issue	Why not auto-fixed	What I did instead
Double submissions (same student + subject within 15 min)	Could be two genuine issues	Only already-Cancelled ones excluded; the rest are kept and counted
"Exam Cell", "Hostel", "Fees", "IT"	Semantic mapping, e.g. is Exam Cell the same team as Academics?	Mapped provisionally and flagged to the Helpdesk Manager; analysis uses the <i>resolving department</i> from the audit log instead
Walk-in tickets with events before "created"	The true creation time is unknowable	Excluded from the KPI (8 tickets) and reported; desk clock to be fixed
Missing priority on email tickets	Changes the SLA target (24h vs 72h)	Policy default (Medium) applied and flagged; KPI recomputed with "High" as sensitivity (section 9)
Ticket-table resolved_at disagrees with the audit log	For reopened tickets it holds the first, not the final, fix	Audit log chosen as source of truth; column ignored; reported to the IT admin
Orphan audit events (unknown ticket_id)	Cannot be attributed	Quarantined, not joined

Assumptions & unresolved limitations

- Semantic category mappings pending owner confirmation: Academics, Exam Cell, Fees, Finance, HOSTEL, Hostel, IT.
- Missing priority (email tickets) = Medium, per draft SLA policy default.
- SLA measured in **calendar** hours (the draft policy). Business-hours SLA is not decided; the sensitivity range is 57.5%–65.9%.
- CSAT is directional only: 36% response, and students whose ticket breached SLA respond more often.
- The data is a daily batch export (last sync 21 Sep 06:00), so it is fit for weekly or term reporting but **not** for a live queue dashboard.

8 Workflow model with data

Source systems are organised by *system*. The question is about the *ticket's journey*. So `pipeline/model.py` builds the **smallest model that explains the workflow**: entities (student, agent, ticket), events (the lifecycle), interactions (student and agent messages), interventions (escalations, reminders) and outcomes (SLA, reopen, CSAT). These are joined into one row per ticket.

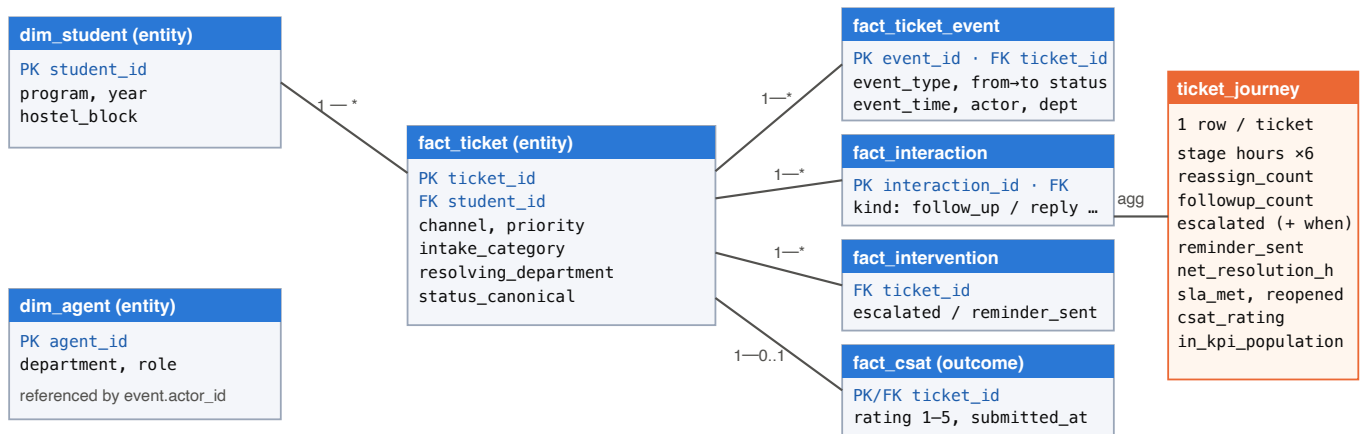


Table	PK	FK	Grain	Rows	Role
dim_student	student_id	—	1 per student	1,800	entity
dim_agent	agent_id	—	1 per agent	15	entity
fact_ticket	ticket_id	student_id	1 per business ticket (deduped)	1,230	entity
fact_ticket_event	event_id	ticket_id, actor_id	1 per lifecycle event	7,732	event / state change
fact_interaction	interaction_id	ticket_id	1 per portal message	4,550	interaction
fact_intervention	event_id	ticket_id	1 per escalation / reminder	853	intervention
fact_csat	ticket_id	ticket_id	1 per rated ticket (latest)	394	outcome
ticket_journey	ticket_id	all of the above, aggregated	1 per ticket	1,230	analysis model

How ticket_journey is built (one-to-many tables are aggregated before joining, to keep the grain)

Field	Derived from
created, first_assigned, first_response, first/final resolved	min/max of the relevant audit events per ticket
reassign_count · reopened · resolving_department	count of reassigned · any → Reopened · department on the last routing event
pause_hours	sum of paired intervals → Waiting on Student ... Waiting on Student →
6 stage durations	triage, first-response wait, re-routing, work, waiting-on-student, reopen cycle; they sum exactly to gross time (checked in code)
followup_count · escalated · reminder_sent · csat_rating	interactions API · interventions · CSAT (left joins)
sla_target · sla_met · in_kpi_population	priority → policy target; net hours ≤ target; exclusion reasons

Three reconstructed journeys (from the audit log)

On time: TKT-100008 · Medium priority (target 72h) · net 16h · **met SLA** · reassignments 0 · follow-ups 1

When	Event	Department	Actor
Mon 27 Jul 17:08	created → New		STU24865
Mon 27 Jul 17:21	assigned → Assigned	IT Support	SYSTEM
Mon 27 Jul 18:24	first_response → In Progress	IT Support	AGT004
Tue 28 Jul 09:25	status_change → Resolved	IT Support	AGT004
Fri 31 Jul 09:25	status_change → Closed	IT Support	SYSTEM

Breached after misrouting: TKT-100002 · Low priority (target 120h) · net 212h · **BREACHED** · reassignments 1 · follow-ups 5

When	Event	Department	Actor
Mon 27 Jul 13:09	created → New		STU25714
Mon 27 Jul 13:22	assigned → Assigned	Fees & Finance	SYSTEM
Tue 28 Jul 13:22	reminder_sent	Fees & Finance	SYSTEM
Thu 30 Jul 10:00	escalated	Fees & Finance	AGT015
Sat 01 Aug 16:50	first_response → In Progress	Fees & Finance	AGT008
Mon 03 Aug 11:10	reassigned → Assigned	Academics & Exams	AGT014
Tue 04 Aug 09:40	agent_response → In Progress	Academics & Exams	AGT010
Wed 05 Aug 09:00	status_change → Resolved	Academics & Exams	AGT010
Sat 08 Aug 09:00	status_change → Closed	Academics & Exams	SYSTEM

High priority, escalated after breach: TKT-100014 · High priority (target 24h) · net 66h · **BREACHED** · reassignments 0 · follow-ups 3

When	Event	Department	Actor
Mon 27 Jul 19:09	created → New		STU24380
Mon 27 Jul 19:13	assigned → Assigned	Fees & Finance	SYSTEM
Tue 28 Jul 19:13	reminder_sent	Fees & Finance	SYSTEM
Wed 29 Jul 09:33	first_response → In Progress	Fees & Finance	AGT008
Thu 30 Jul 10:00	escalated	Fees & Finance	AGT015
Thu 30 Jul 13:02	status_change → Resolved	Fees & Finance	AGT008
Sun 02 Aug 13:02	status_change → Closed	Fees & Finance	SYSTEM

Metrics are computed in SQL over the model (joins + aggregations)

pipeline/metrics.py

```
"by_reassignment": ""
SELECT CASE WHEN reassign_count = 0 THEN '0 (routed right first time)'
           WHEN reassign_count = 1 THEN '1 reassignment' ELSE '2+ reassignments' END AS segment,
COUNT(*) AS tickets, ROUND(100.0 * AVG(sla_met), 1) AS sla_pct,
ROUND(MEDIAN(net_resolution_hours), 1) AS median_net_h,
ROUND(AVG(followup_count), 2) AS avg_followups,
ROUND(100.0 * AVG(reopened), 1) AS reopen_pct
FROM ticket_journey WHERE in_kpi_population = 1 GROUP BY 1 ORDER BY 1""",
```

pipeline/metrics.py

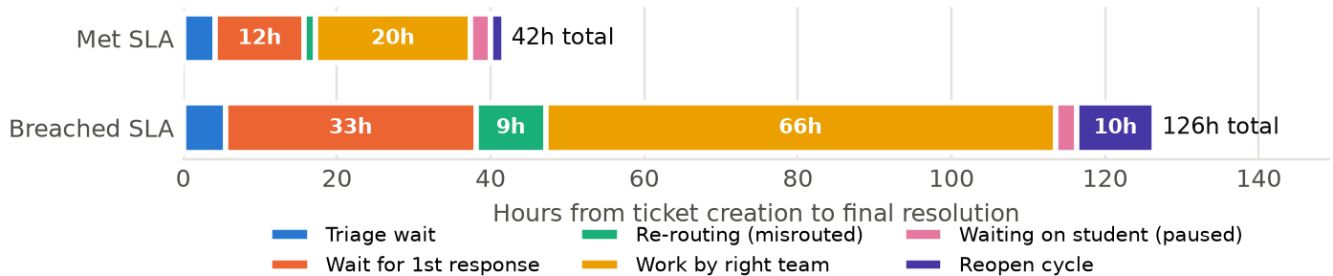
```
"followups_vs_outcome": ""
SELECT CASE WHEN sla_met = 1 THEN 'met SLA' ELSE 'breached SLA' END AS segment, COUNT(*) AS tickets,
ROUND(AVG(followup_count), 2) AS avg_followups,
ROUND(AVG(c.rating), 2) AS avg_csats, COUNT(c.rating) AS csats_n
FROM ticket_journey j LEFT JOIN fact_csats c ON c.ticket_id = j.ticket_id
WHERE in_kpi_population = 1 GROUP BY 1""",
```

A MEDIAN aggregate is registered on the SQLite connection so medians are computed in SQL as well. Every segment table is saved to output/runs/<run_id>/segments/.

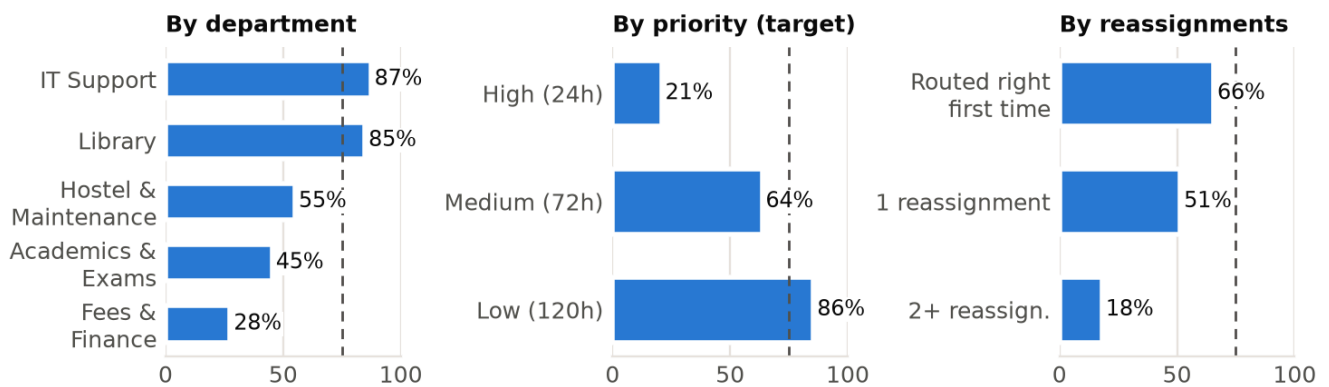
9 Findings

So what? Breached tickets take **126h** on average vs **42h** for tickets that met SLA. The gap is mostly **waiting for a first response (33h vs 12h)**, **time inside slow teams (66h vs 20h)** and **re-routing (9h vs 2h)**. Waiting on the student is the same for both (about 3h), so students are not the cause.

Where the time goes (average hours per ticket)



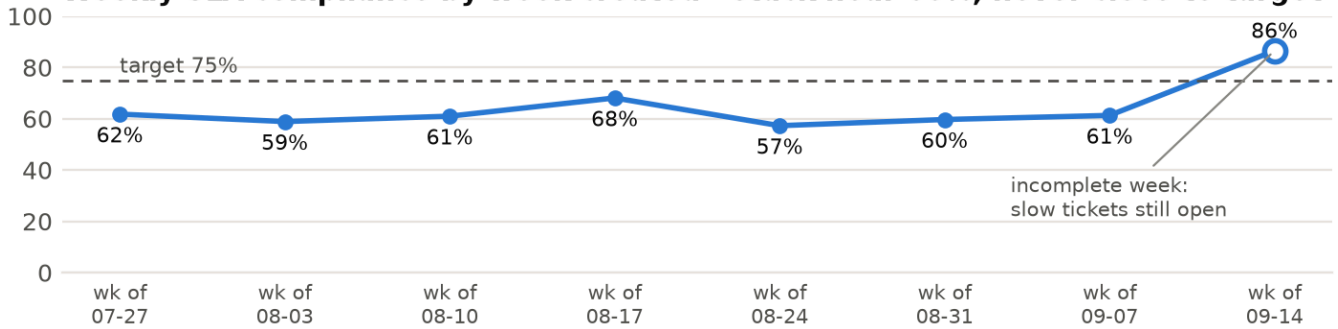
SLA compliance, % of resolved tickets (dashed line = 75% target)



Hypothesis	Verdict	Evidence
H1 Misrouting drives breaches	SUPPORTED	SLA 66% (0 hops) → 51% (1) → 18% (2+); breached tickets spend 9h re-routing vs 2h
H1b Manual triage alone drives breaches	PARTLY	Manual queue waits 13.2h, but SLA is 61% vs 65% auto-routed; email 63% ≈ portal 62%. The wait is real but small next to department queues.
H2 Priority not used in queueing	SUPPORTED	Median first response High 19.3h ≈ Low 16.3h; SLA High 21% · Medium 64% · Low 86%
H3 Capacity / hours constrained teams	SUPPORTED	Fees & Finance 28%, Academics 45% (Mon–Fri 9–17) vs IT 87%, Library 85%
H4 Escalation too late	SUPPORTED	Median escalation at 65h (before breach) / 86h (after); 132 escalated after SLA passed, of which 1% met
H5 Weekend tickets are the main problem	MINOR	Weekend 56% vs weekday 65%: a real gap, but only 206 tickets; a secondary driver

Association ≠ causation. Escalated tickets breach more (89% SLA when not escalated) partly because *only slow tickets get escalated*. Likewise, tickets with an auto-reminder (32% vs 75%) were already slow. The data shows escalation happens late. It cannot prove how much earlier escalation would help, which is why S3 below is a pilot, not a promise.

Weekly SLA compliance by week created - stuck near 60%, never close to target



Students feel it

	Avg follow-ups	Avg CSAT
Met SLA	1.05	4.37
Breached SLA	3.79	2.86

Every breach creates about 2.7 extra "any update?" messages, which is load the agents create for themselves. A chatbot could answer these messages, but would not remove the cause.

Is the KPI robust to its definition?

Definition	SLA %
A. Chosen: final resolution, net of waiting-on-student, missing priority = Medium	63.1
B. Gross calendar hours (no pause for waiting-on-student)	61.7
C. First resolution instead of final (ignores reopens)	65.9
D. Missing priority treated as High (24h)	57.5
E. Also count still-open tickets already past deadline as breaches	60.9

Definitions move the number by up to 8.4 pp but **none changes the conclusion**. The Dean must sign off one definition **before** the pilot so before and after are comparable.

Recommendation (Situation → Complication → Resolution)

Situation: the helpdesk receives about 153 tickets a week across 5 teams with a draft 24/72/120h SLA. **Complication:** only 63% meet it (21% of High-priority), and the proposed fix, a chatbot, targets first contact, which is not where the time goes. **Resolution:** fix routing, queueing and escalation inside the current tool first. Revisit the chatbot for FAQ-style tickets once the process is sound.

Scenario	Change	Key assumption	Est. SLA %	Δ pp	Est. High %
Baseline (today)	-	measured	63.1	0.0	20.9
S1 Fix intake routing	Email/'Other' tickets get a category at intake (portal form link + keyword suggestion); removes manual-triage wait and re-routing time	removed time is not replaced by other waiting	68.6	5.5	26.6
S2 Priority-first queue	High-priority tickets jump the department queue	High first-response wait falls 80% (to ~3h)	64.6	1.5	32.4
S3 Escalate at 50% of SLA	Replace 'open > 48h at 10:00 daily review' with an automatic alert at 50% of the ticket's SLA	escalation cuts remaining time by 40% - NOT measured, must be piloted	72.9	9.8	29.5
S1 + S2		as above	70.2	7.1	38.8
S1 + S2 + S3		as above	78.8	15.7	48.9

Estimates re-score each measured ticket journey with a stage removed or shrunk (analysis/what_if.py). S1 and S2 are grounded in measured stage times. S3's 40% effect is an **assumption** to be tested in the pilot. The combined estimate (about 79%) clears the 75% target with a margin. High-priority still reaches only about 49%, so High tickets also need a same-day work commitment, not just a faster first response.

Pragmatic action plan (pilot first, then scale)

Initiative	What exactly	Owner	When	Leading indicator
1. Structured intake (S1)	Auto-reply to every email with a portal form link; keyword category suggestion; reassign requires a reason code	Helpdesk Manager + IT admin	Week 1–2 pilot (email channel)	Manual-triage wait <2h; reassignment <6%
2. SLA-based escalation (S3)	Replace the 48h daily review with an automatic alert at 50% of each ticket's SLA (12h for High)	Helpdesk Manager	Week 1–2 pilot (Finance + Academics)	>90% of breaches-at-risk escalated before deadline
3. Priority-first queue (S2)	Queue view sorted by SLA deadline, not arrival; High tickets get a same-day owner	Dept leads	Week 3	High first response <4h
4. Confirm before close	"Did this fix it?" prompt before auto-close; track reopens	IT admin	Week 3–4	Reopen rate not rising
5. Scale + SOP	Roll out to all teams; document triage and escalation SOPs; keep the pipeline running weekly	Helpdesk Manager	Week 4–8	Weekly KPI from the pipeline

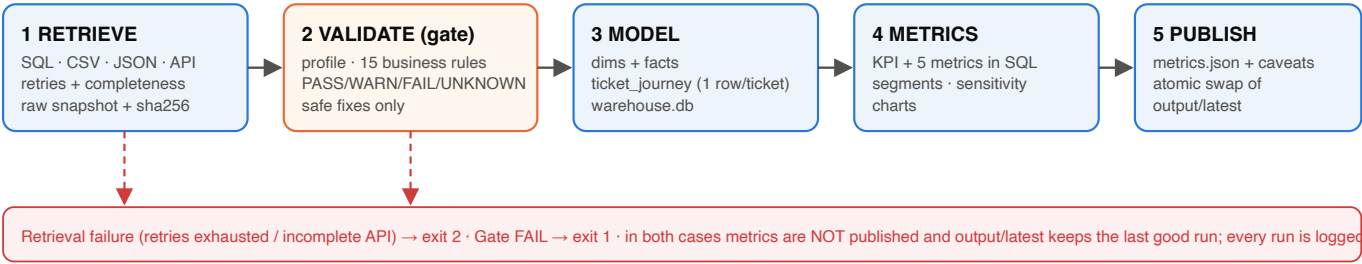
Key success factors

- Dean signs off one SLA definition before the pilot
- Agents accept deadline-sorted queues (brief them on why)
- Reason codes are actually filled in (make the field mandatory)
- The weekly pipeline gate stays PASS or has known WARNs only

Risks

- "Fast but wrong" fixes; guarded by the reopen rate and CSAT
- Escalation fatigue if alerts are too many; tune the threshold in the pilot
- Finance and Academics capacity may still cap the gains; escalate a staffing discussion only with pilot evidence

10 Dependable pipeline



What makes it repeatable

- One command: `python pipeline/run_pipeline.py`; deterministic seed; a `run_id` per run
- Immutable raw snapshot plus manifest (rows, sha256) per run
- Rules as code, the same checks every run; results saved as JSON and Markdown
- Structured log per run: `time | level | stage | message`
- `run_status.json` with per-stage timing and exit code
- **Atomic publish:** `output/latest` is replaced only after a full, gate-passing run (copy then rename)
- 5 pytest tests: parsers, mappings, the SQL median, and the published model contract

Runs performed for this report

Run	Scenario	Gate	Exit code	Published?
demo_run	normal	PASS WITH CAVEATS	0	yes: output/latest replaced, status PROVISIONAL
demo_fail_api_outage	Portal API returns 503 on every call	n/a (stopped in retrieve)	2	no: output/latest untouched
demo_fail_truncated_events	audit-log export cut to 50%	FAIL (R06: 50% lifecycle coverage)	1	no: output/latest untouched

Failure handling

Failure	Detected by	Behaviour
API 500 / 503 / connection error	HTTP status / URLError	Exponential back-off 0.2→3.2s, 6 attempts per page, then stage failure (exit 2)
API 429 rate limit	HTTP 429 + Retry-After	Wait as instructed, retry
API silently incomplete	fetches ≠ total_records or duplicate ids	Stage failure; nothing published
Missing source / empty file	R01	Gate FAIL (exit 1)
Partial export	R06 lifecycle coverage < 98%	Gate FAIL (exit 1)
Grain broken (conflicting ticket ids)	R02	Gate FAIL
Unknown status / unmapped category	R08 / R09	Gate FAIL, so the mapping table must be updated consciously
Data issues within tolerance	WARN / UNKNOWN	Published as PROVISIONAL; caveats written into metrics.json
Model inconsistency	grain + stage-sum checks after modelling	Logged as WARNING; unit tests fail in CI

Failure run logs (simulated)

```
18:41:55 | WARNING | retrieve | API HTTP 503 on attempt 1/6 -> retry in 0.2s (page 1)
18:42:01 | ERROR | retrieve | API HTTP 503 on attempt 6/6 -> giving up (page 1)
18:42:01 | ERROR | run | STAGE FAILURE: API retries exhausted for (page 1) -> run aborted, metrics NOT
published, output/latest left unchanged
18:42:01 | INFO | run | === run finished, exit code 2 ===

18:42:02 | WARNING | retrieve | SIMULATION: ticket_events.csv export truncated to 50%
18:42:08 | ERROR | validate | R06 FAIL Every ticket has a 'created' event and both systems agree on created_at ::
tickets with audit-log lifecycle: 50.0%; created_at agreement within 1 min: 99.5% (DD/MM rows agree only when parsed
day-first)
18:42:08 | INFO | validate | VALIDATION GATE: FAIL
18:42:08 | ERROR | validate | gate FAILED on ['R06'] -> metrics NOT published; output/latest left unchanged
18:42:08 | INFO | run | === run finished, exit code 1 ===
```

Normal run: validate → model → metrics → publish (log excerpt)

```
18:41:48 | INFO | run | === CampusDesk pipeline run demo_run ===
18:41:48 | INFO | run | started mock Portal API (normal mode)
18:41:54 | INFO | validate | profiled tickets=1254 events=7849 interactions=4550 csat=424
18:41:54 | INFO | validate | VALIDATION GATE: PASS WITH CAVEATS
18:41:54 | INFO | metrics | KPI SLA compliance = 63.1% (675/1069 resolved tickets)
18:41:54 | INFO | metrics | metric M1_median_first_response_h = 17.9
18:41:54 | INFO | metrics | metric M2_median_manual_triage_wait_h = 13.2
18:41:54 | INFO | metrics | metric M3_reassignment_rate_pct = 12.4
18:41:54 | INFO | metrics | metric M4_reopen_rate_pct = 9.4
18:41:54 | INFO | metrics | metric M5_avg_csat = 3.69
18:41:54 | INFO | metrics | metric M5_csat_response_rate_pct = 36.5
18:41:55 | INFO | publish | published run demo_run to output/latest (status=PROVISIONAL)
18:41:55 | INFO | run | === run finished, exit code 0 ===
```

Reliability evidence: does the pipeline get the right answer?

Because the data is synthetic, the generator's hidden ground truth lets me **prove** the pipeline is correct. That isn't possible with real data. tests/reconcile_with_truth.py compares:

Method	Tickets used	SLA %	Error vs truth (pp)
Ground truth (simulated)	1077	63.5	0.0
Pipeline (cleaned + modelled)	1069	63.1	-0.4
Naive query on ticket table	714	64.6	1.1

The pipeline is within **0.4 pp** of the truth and agrees on 99% of individual ticket verdicts (the differences are tickets whose missing priority was defaulted). A **naive query** on the ticket table looks plausible (64.6%), but it silently uses only 714 tickets. It drops DD/MM dates, status spellings and missing priorities, keeps duplicates, and times reopened tickets to their *first* fix. It is close only by accident: its errors partly cancel out, and its segment-level numbers would be wrong.

Two real bugs were caught this way while building. (1) A clock-skewed walk-in ticket that the first version of R07 missed: the rule was widened from "resolved before created" to "any event before created". (2) Reopened tickets that were still open were counted as resolved, now excluded by status. This is why the check exists.

11 What remains unknown, and what I'd do next

Unknown / limitation	Impact on conclusions	How to resolve	Owner
SLA definition not signed off (calendar vs business hours, pause rule)	KPI ranges 57.5–65.9%; conclusions unchanged	Sign-off meeting before the pilot	Dean
Why tickets are misrouted	We know the cost, not the cause	Mandatory reassign reason code; 2 weeks of data	Helpdesk Manager
Real agent effort vs queueing inside a team	"Work" time mixes both, so the capacity case is unproven	"Start work" status + effort field	IT admin
Causal effect of earlier escalation	S3 uplift is an assumption	A/B pilot: Finance and Academics vs other teams	FDE + Manager
CSAT non-response bias (36% response)	Satisfaction level uncertain; direction is clear	Shorter 1-click survey in the resolution email	Helpdesk Manager
Walk-in desk clock skew; stale resolved_at column	8 tickets excluded; column ignored	Fix the desk PC time sync; fix the denormalisation job	IT admin
Synthetic data	Patterns are plausible but simulated	Run the same pipeline on the real export (same schema contract)	FDE

Would I build the AI chatbot now? No. First fix routing and escalation, and sign off the KPI definition. Once the process is sound, a chatbot is worth testing for FAQ-type tickets (e.g. Wi-Fi, LMS login) and for the "any update?" follow-ups, measured with this same pipeline.

12 5-minute demo script

Time	Show	Say
0:00–0:40	This PDF, cover page	"The Dean wanted to buy an AI chatbot because tickets take forever. As an FDE I didn't build the request. I first found where the time goes."
0:40–1:30	Problem brief + workflow map (p.2, p.4)	Stakeholders; the workflow; 5 pain points; SMART statement: 63% → 75% in 8 weeks with the same team and tool.
1:30–2:30	Terminal: python pipeline/run_pipeline.py	4 source types; the API fails with 500s and 429s and the pipeline retries and proves all 4,550 records arrived; 15 rules, PASS/WARN/UNKNOWN; gate passes with caveats; KPI 63.1%.
2:30–3:00	Terminal: --simulate truncated_events	"Half the audit log is missing, so R06 fails and nothing is published. output/latest still has the last good run."
3:00–4:10	Findings charts (p.12–13)	Priority ignored (High 21%); misrouting 66→18%; Finance and Academics; escalation at 65h. Association vs causation.
4:10–4:40	Recommendation + what-if (p.13–14)	Fix routing and escalation first (about 79% estimated), pilot 2 weeks, leading indicators; no chatbot yet.
4:40–5:00	Reconciliation + unknowns (p.16–17)	Pipeline within 0.4 pp of truth, while the naive query is off; what remains unknown and who owns it.

Repo: [GitHub link \(to be added\)](#) . Full executed walkthrough: notebooks/CampusDesk_FDE_Walkthrough.ipynb (also exported as HTML).